76ac2d35-89a2-4cac-a434-182139b04a56

<prev_user>

<request>

You are working on an issue in the 'Janpot/split-ui' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/split-ui/split-ui (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2025-11-06T11:52:38.805Z</current_datetime>

Consider the following problem statement:

<problem_statement>
Read https://motion.dev/blog/web-animation-performance-tier-list carefully, make recommendations for the resizer implementation. We don;t want to install motion, just apply the knowledge of the blogpost to our implementation
</problem_statement>

Implement the necessary changes to the repository so that the requirements specified in the <problem_statement> are met.

Ensure that you lint, build and test early and iteratively to validate each code change thoroughly.

<prev_assistant>

Perfect! Let me create one final comprehensive summary:

Summary

I have successfully implemented performance optimizations for the resizer component in the split-ui library based on the recommendations from the Motion.dev blog post about web animation performance.

What Was Done

1. **Performance Optimizations Implemented**

RequestAnimationFrame Throttling:
- Pointer move events are now throttled to ~60fps using `requestAnimationFrame`
- Reduces layout calculations from 100s/second to ~60/second
- Implements natural event coalescing (latest offset used, intermediate values dih optimization
- Performance measurement guidelines
- Before/after comparison
- Context on why transform (S-tier) isn't used
- Best practices for developers

Inline Documentation:
- File header explaining all optimizations
- JSDoc comments for DragState properties
- Comments explaining lifecycle and behavior

3. **Quality Assurance**

✅ Build successful
✅ Linting passed
✅ Type checking passed
✅ All code review comments addressed
✅ Security scan passed (0 vulnerabilities)
✅ Bundle size: +1.17 kB (acceptable for performance gains)

Key Insight

While the Motion.dev blog recommends S-tier properties (transform, opacity) for animations, resizing is fundamentally a layout operation that must actually change element dimensions. The optimizations apply the blog's core principles while working within the constraints of a layout-changing operation:

- Minimize layout thrashing via RAF
- Provide browser hints via will-change
- Limit layout scope via CSS contain
- Defer non-critical work
- Natural event coalescing

Files Changed

- `packages/react/src/core.ts`: Performance optimizations + documentation
- `packages/react/src/public/styles.css`: CSS containment properties
- `packages/react/PERFORMANCE.md`: Comprehensive performance guide

The implementation is production-ready and maintains backward compatibility while significantly improving resize performance.

<current>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'Janpot/split-ui' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/split-ui/split-ui (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2025-11-06T12:31:47.795Z</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
Optimize resizer performance with RAF throttling and CSS containment
</pr_title>

<pr_description>
Applied web animation performance best practices from Motion.dev's tier list to minimize layout thrashing during

````
resize operations.

## Changes

**RequestAnimationFrame throttling**
- Pointer move events throttled to ~60fps via RAF (down from
100s+/second)
- Natural event coalescing: latest offset used, intermediate
values discarded
- State cleanup on drag end prevents stale RAF callbacks

**Performance hints**
- `will-change: flex-basis` applied during drag, cleared on
completion
- Allows browser to pre-optimize before layout changes occur

**CSS containment**
- `contain: layout style` on panels limits recalculation scope
- `contain: layout style paint` on resizers provides full
isolation

**Deferred updates**
- ARIA attributes updated only on commit, not during drag

## Implementation

```typescript
interface DragState {
  rafId?: number;        // Schedule one update per frame
  pendingOffset?: number; // Latest value wins, older discarded
}

function handlePointerMove(event: PointerEvent) {
  currentDragState.pendingOffset = getOffset(event);
  if (!currentDragState.rafId) {
    currentDragState.rafId =
requestAnimationFrame(updateLayout);
  }
}
```

## Note on S-tier properties

Transform/opacity not used—resizing requires actual layout
changes to element dimensions. Optimizations focus on minimizing
impact of necessary layout operations.
````

Added `PERFORMANCE.md` documenting approach, measurement guidelines, and rationale.

Bundle size: +1.17 kB

> [!WARNING]
>
>

</pr_description>

Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>

<comment_new>
<comment_id>3497046072</comment_id>
<author>@Janpot</author>
@copilot Seems like a bad idea, we're updating the flex value, which would always cause layout recalculation. how is will-change helping here?
</comment_new>

</pr_comments>

</comments>

The last **6** git commits in this branch are the changes you have made so far. Use those as your change commit history.
</problem_statement>

Analyze and determine if action or explanation is needed
<rules_for_making_changes>
* Start by fully understanding <problem_statement> and the <comments> before making any changes. You will only address the <comment_new> comments. Reply to new comments as you address them. Ignore comments that are not actionable or directed at you.

```
* For each new comment, determine:
    - If it a request for changes, a question on the code or a
suggestion.
    - Is it addressed to you or someone else, e.g. "Hey
@username, ..." or "@username can you ...".
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can
stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and
ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>

Ensure that you lint, build and test early and iteratively to
validate each code change thoroughly.
```